1. ADDITION

additiontest.java

```
package unittest;

public class additiontest {
   public int add(int a, int b){
      return a + b;
   }
}
```

. Test Class

Additiontestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Additiontestcase extends TestCase {

   public void testAddition() {
      additiontest t = new additiontest();
      int output = t.add(-8, 2);
      assertEquals(-6, output);
   }
}
```

---

2. SUBTRACTION

subtractiontest.java

```
package unittest;

public class subtractiontest {
   public int subtract(int a, int b){
      return a - b;
   }
}
```

Subtractiontestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Subtractiontestcase extends TestCase {
```

```
  public void testSubtraction() {
    subtractiontest t = new subtractiontest();
    int output = t.subtract(10, 4);
    assertEquals(6, output);
  }
}
```

🔹 3. MULTIPLICATION

📄 **multiplicationtest.java**

```
package unittest;

public class multiplicationtest {
  public int multiply(int a, int b){
    return a * b;
  }
}
```

📄 **Multiplicationtestcase.java**

```
package unittest;

import junit.framework.TestCase;

public class Multiplicationtestcase extends TestCase {

  public void testMultiplication() {
    multiplicationtest t = new multiplicationtest();
    int output = t.multiply(5, 3);
    assertEquals(15, output);
  }
}
```

🔹 4. DIVISION

📄 **divisiontest.java**

```
package unittest;

public class divisiontest {
  public int divide(int a, int b){
    return a / b;
  }
}
```

📄 **Divisiontestcase.java**

```
package unittest;

import junit.framework.TestCase;

public class Divisiontestcase extends TestCase {

  public void testDivision() {
    divisiontest t = new divisiontest();
    int output = t.divide(10, 2);
    assertEquals(5, output);
  }
}
```

🔹 5. SQUARE

📄 **squaretest.java**

```
package unittest;

public class squaretest {
  public int square(int a){
    return a * a;
  }
}
```

📄 **Squaretestcase.java**

```
package unittest;

import junit.framework.TestCase;

public class Squaretestcase extends TestCase {

  public void testSquare() {
    squaretest t = new squaretest();
    int output = t.square(4);
    assertEquals(16, output);
  }
}
```

🔹 6. SALARY

📄 **salarytest.java**

```
package unittest;

public class salarytest {
  public double calculateSalary(double basic){
```

```
    double hra = 0.2 * basic;
    double da = 0.1 * basic;
    return basic + hra + da;
  }
}
```

Salarytestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Salarytestcase extends TestCase {

  public void testSalary() {
    salarytest t = new salarytest();
    double output = t.calculateSalary(10000);
    assertEquals(13000.0, output);
  }
}
```

---

1. PRIME NUMBER

primetest.java

```
package unittest;

public class primetest {
  public String checkPrime(int n){
    if(n < 1 || n > 100){
      return "Invalid";
    }

    if(n == 1){
      return "Not Prime";
    }

    for(int i = 2; i <= n/2; i++){
      if(n % i == 0){
        return "Not Prime";
      }
    }
    return "Prime";
  }
}
```

Primetestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Primetestcase extends TestCase {

  public void testPrime() {
    primetest t = new primetest();
    String output = t.checkPrime(7);
    assertEquals("Prime", output);
  }
}
```

🔥 2. POWER (a^b)

📄 powertest.java

```
package unittest;

public class powertest {
  public int power(int a, int b){
    if(a < 1 || a > 10 || b < 1 || b > 5){
      return -1;
    }

    int result = 1;
    for(int i = 0; i < b; i++){
      result = result * a;
    }
    return result;
  }
}
```

📄 Powertestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Powertestcase extends TestCase {

  public void testPower() {
    powertest t = new powertest();
    int output = t.power(2, 3);
    assertEquals(8, output);
  }
}
```

🔥 3. LARGEST OF 3 NUMBERS

📄 largesttest.java

```
package unittest;

public class largesttest {
  public int largest(int a, int b, int c){
    if(a < 1 || b < 1 || c < 1 || a > 50 || b > 50 || c > 50){
      return -1;
    }

    if(a >= b && a >= c){
      return a;
    }
    else if(b >= a && b >= c){
      return b;
    }
    else{
      return c;
    }
  }
}
```

📄 Largesttestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Largesttestcase extends TestCase {

  public void testLargest() {
    largesttest t = new largesttest();
    int output = t.largest(10, 20, 30);
    assertEquals(30, output);
  }
}
```

🔥 4. ANGLE QUADRANT

📄 angletest.java

```
package unittest;

public class angletest {
  public String findQuadrant(int angle){
    if(angle < 0 || angle > 360){
      return "Invalid";
```

```
    }

    if(angle > 0 && angle < 90){
      return "Q1";
    }
    else if(angle > 90 && angle < 180){
      return "Q2";
    }
    else if(angle > 180 && angle < 270){
      return "Q3";
    }
    else if(angle > 270 && angle < 360){
      return "Q4";
    }
    else{
      return "Axis";
    }
  }
}
```

📄 Angletestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Angletestcase extends TestCase {

  public void testAngle() {
    angletest t = new angletest();
    String output = t.findQuadrant(45);
    assertEquals("Q1", output);
  }
}
```

🔥 5. PLAYER RECORD

📄 playertest.java

```
package unittest;

public class playertest {
  public String player(String name, String team, int avg){

    if(name.length() > 30 || team.length() > 20 || avg < 0){
      return "Invalid";
    }
```

```
    return team + " : " + name + " - " + avg;
  }
}
```

Playertestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Playertestcase extends TestCase {

  public void testPlayer() {
    playertest t = new playertest();
    String output = t.player("Dhoni", "CSK", 50);
    assertEquals("CSK : Dhoni - 50", output);
  }
}
```

---

6. EVEN OR ODD

evenoddtest.java

```
package unittest;

public class evenoddtest {
  public String check(int n){
    if(n % 2 == 0){
      return "Even";
    }
    else{
      return "Odd";
    }
  }
}
```

Evenoddtestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Evenoddtestcase extends TestCase {

  public void testEvenOdd() {
    evenoddtest t = new evenoddtest();
    String output = t.check(4);
```

```
    assertEquals("Even", output);
  }
}
```

🔥 7. FACTORIAL

📄 factorialtest.java

```
package unittest;

public class factorialtest {
  public int fact(int n){
    if(n < 0){
      return -1;
    }

    int f = 1;
    for(int i = 1; i <= n; i++){
      f = f * i;
    }
    return f;
  }
}
```

📄 Factorialtestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Factorialtestcase extends TestCase {

  public void testFactorial() {
    factorialtest t = new factorialtest();
    int output = t.fact(5);
    assertEquals(120, output);
  }
}
```

🔥 8. PALINDROME NUMBER

📄 palindrometest.java

```
package unittest;

public class palindrometest {
  public String check(int n){
    int temp = n, rev = 0;
```

```
    while(n > 0){
      int r = n % 10;
      rev = rev * 10 + r;
      n = n / 10;
    }

    if(temp == rev){
      return "Palindrome";
    }
    else{
      return "Not Palindrome";
    }
  }
}
```

📄 Palindrometestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Palindrometestcase extends TestCase {

  public void testPalindrome() {
    palindrometest t = new palindrometest();
    String output = t.check(121);
    assertEquals("Palindrome", output);
  }
}
```

🔥 9. SIMPLE INTEREST

📄 interesttest.java

```
package unittest;

public class interesttest {
  public double calculate(double p, double r, double t){
    return (p * r * t) / 100;
  }
}
```

📄 Interesttestcase.java

```
package unittest;

import junit.framework.TestCase;
```

```
public class Interesttestcase extends TestCase {

  public void testInterest() {
    interesttest t = new interesttest();
    double output = t.calculate(1000, 5, 2);
    assertEquals(100.0, output);
  }
}
```

🔥 10. MAX OF TWO NUMBERS

📄 maxtest.java

```
package unittest;

public class maxtest {
  public int max(int a, int b){
    if(a > b){
      return a;
    }
    else{
      return b;
    }
  }
}
```

📄 Maxtestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Maxtestcase extends TestCase {

  public void testMax() {
    maxtest t = new maxtest();
    int output = t.max(10, 20);
    assertEquals(20, output);
  }
}
```

---

11. SUM OF DIGITS

📄 sumdigitstest.java

```
package unittest;

public class sumdigitstest {
```

```
  public int sum(int n){
    int s = 0;

    while(n > 0){
      s = s + (n % 10);
      n = n / 10;
    }
    return s;
  }
}
```

📄 Sumdigitstestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Sumdigitstestcase extends TestCase {

  public void testSumDigits() {
    sumdigitstest t = new sumdigitstest();
    int output = t.sum(123);
    assertEquals(6, output);
  }
}
```

🔥 12. REVERSE NUMBER

📄 reversetest.java

```
package unittest;

public class reversetest {
  public int reverse(int n){
    int rev = 0;

    while(n > 0){
      rev = rev * 10 + (n % 10);
      n = n / 10;
    }
    return rev;
  }
}
```

📄 Reversetestcase.java

```
package unittest;

import junit.framework.TestCase;
```

```
public class Reversetestcase extends TestCase {

   public void testReverse() {
      reversetest t = new reversetest();
      int output = t.reverse(123);
      assertEquals(321, output);
   }
}
```

🔥 13. VOWEL OR CONSONANT

📄 voweltest.java

```
package unittest;

public class voweltest {
   public String check(char ch){
      if(ch=='a'||ch=='e'||ch=='i'||ch=='o'||ch=='u'||
        ch=='A'||ch=='E'||ch=='I'||ch=='O'||ch=='U'){
         return "Vowel";
      }
      else{
         return "Consonant";
      }
   }
}
```

📄 Voweltestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Voweltestcase extends TestCase {

   public void testVowel() {
      voweltest t = new voweltest();
      String output = t.check('a');
      assertEquals("Vowel", output);
   }
}
```

🔥 14. LEAP YEAR

📄 leapyeartest.java

```
package unittest;

public class leapyeartest {
  public String check(int year){
    if((year % 4 == 0 && year % 100 != 0) || year % 400 == 0){
      return "Leap Year";
    }
    else{
      return "Not Leap Year";
    }
  }
}
```

📄 Leapyeartestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Leapyeartestcase extends TestCase {

  public void testLeapYear() {
    leapyeartest t = new leapyeartest();
    String output = t.check(2024);
    assertEquals("Leap Year", output);
  }
}
```

🔥 15. GRADE CALCULATION

📄 gradetest.java

```
package unittest;

public class gradetest {
  public String grade(int marks){
    if(marks >= 90) return "A";
    else if(marks >= 75) return "B";
    else if(marks >= 50) return "C";
    else return "Fail";
  }
}
```

📄 Gradetestcase.java

```
package unittest;

import junit.framework.TestCase;
```

```
public class Gradetestcase extends TestCase {

  public void testGrade() {
    gradetest t = new gradetest();
    String output = t.grade(85);
    assertEquals("B", output);
  }
}
```

🔥 16. SWAP TWO NUMBERS

📄 swaptest.java

```
package unittest;

public class swaptest {
  public int[] swap(int a, int b){
    int temp = a;
    a = b;
    b = temp;
    return new int[]{a, b};
  }
}
```

📄 Swaptestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Swaptestcase extends TestCase {

  public void testSwap() {
    swaptest t = new swaptest();
    int[] output = t.swap(5, 10);
    assertEquals(10, output[0]);
    assertEquals(5, output[1]);
  }
}
```

---

17. CHECK POSITIVE / NEGATIVE

📄 positivetest.java

```
package unittest;

public class positivetest {
  public String check(int n){
```

```
    if(n > 0) return "Positive";
    else if(n < 0) return "Negative";
    else return "Zero";
  }
}
```

📄 Positivetestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Positivetestcase extends TestCase {

  public void testPositive() {
    positivetest t = new positivetest();
    String output = t.check(5);
    assertEquals("Positive", output);
  }
}
```

🔥 18. SUM OF ARRAY

📄 arraysumtest.java

```
package unittest;

public class arraysumtest {
  public int sum(int arr[]){
    int s = 0;
    for(int i = 0; i < arr.length; i++){
      s += arr[i];
    }
    return s;
  }
}
```

📄 Arraysumtestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Arraysumtestcase extends TestCase {

  public void testArraySum() {
    arraysumtest t = new arraysumtest();
    int[] arr = {1,2,3,4};
    int output = t.sum(arr);
```

```
    assertEquals(10, output);
  }
}
```

🔥 19. FIND MINIMUM IN ARRAY

📄 mintest.java

```
package unittest;

public class mintest {
  public int min(int arr[]){
    int m = arr[0];
    for(int i = 1; i < arr.length; i++){
      if(arr[i] < m){
        m = arr[i];
      }
    }
    return m;
  }
}
```

📄 Mintestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Mintestcase extends TestCase {

  public void testMin() {
    mintest t = new mintest();
    int[] arr = {5,2,8,1};
    int output = t.min(arr);
    assertEquals(1, output);
  }
}
```

🔥 20. COUNT DIGITS

📄 countdigitstest.java

```
package unittest;

public class countdigitstest {
  public int count(int n){
    int c = 0;
```

```
    while(n > 0){
      c++;
      n = n / 10;
    }
    return c;
  }
}
```

📄 Countdigitstestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Countdigitstestcase extends TestCase {

  public void testCountDigits() {
    countdigitstest t = new countdigitstest();
    int output = t.count(12345);
    assertEquals(5, output);
  }
}
```

🔥 21. CHECK ARMSTRONG NUMBER

📄 armstrongtest.java

```
package unittest;

public class armstrongtest {
  public String check(int n){
    int temp = n, sum = 0;

    while(n > 0){
      int r = n % 10;
      sum += r*r*r;
      n = n / 10;
    }

    if(sum == temp) return "Armstrong";
    else return "Not Armstrong";
  }
}
```

📄 Armstrongtestcase.java

```
package unittest;

import junit.framework.TestCase;
```

```
public class Armstrongtestcase extends TestCase {

  public void testArmstrong() {
    armstrongtest t = new armstrongtest();
    String output = t.check(153);
    assertEquals("Armstrong", output);
  }
}
```

🔥 22. MULTIPLICATION TABLE (return single value)

📄 tabletest.java

```
package unittest;

public class tabletest {
  public int table(int n, int i){
    return n * i;
  }
}
```

📄 Tabletestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Tabletestcase extends TestCase {

  public void testTable() {
    tabletest t = new tabletest();
    int output = t.table(5, 3);
    assertEquals(15, output);
  }
}
```

---

23. CHECK NUMBER IN RANGE

📄 rangetest.java

```
package unittest;

public class rangetest {
  public String check(int n){
    if(n >= 1 && n <= 100){
      return "In Range";
    }
```

```
    else{
      return "Out of Range";
    }
  }
}
```

📄 Rangetestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Rangetestcase extends TestCase {

  public void testRange() {
    rangetest t = new rangetest();
    String output = t.check(50);
    assertEquals("In Range", output);
  }
}
```

🔥 24. COUNT VOWELS IN STRING

📄 countvowelstest.java

```
package unittest;

public class countvowelstest {
  public int count(String str){
    int c = 0;

    for(int i = 0; i < str.length(); i++){
      char ch = str.charAt(i);
      if(ch=='a'||ch=='e'||ch=='i'||ch=='o'||ch=='u'||
        ch=='A'||ch=='E'||ch=='I'||ch=='O'||ch=='U'){
         c++;
      }
    }
    return c;
  }
}
```

📄 Countvowelstestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Countvowelstestcase extends TestCase {
```

```
  public void testCountVowels() {
    countvowelstest t = new countvowelstest();
    int output = t.count("Reena");
    assertEquals(3, output);
  }
}
```

🔥 25. STRING LENGTH CHECK

📄 stringlengthtest.java

```
package unittest;

public class stringlengthtest {
  public String check(String str){
    if(str.length() > 5){
      return "Long";
    }
    else{
      return "Short";
    }
  }
}
```

📄 Stringlengthtestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Stringlengthtestcase extends TestCase {

  public void testLength() {
    stringlengthtest t = new stringlengthtest();
    String output = t.check("HelloWorld");
    assertEquals("Long", output);
  }
}
```

🔥 26. CHECK DIVISIBLE BY 5

📄 divisibletest.java

```
package unittest;

public class divisibletest {
  public String check(int n){
```

```
    if(n % 5 == 0){
      return "Divisible";
    }
    else{
      return "Not Divisible";
    }
  }
}
```

📄 Divisibletestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Divisibletestcase extends TestCase {

  public void testDivisible() {
    divisibletest t = new divisibletest();
    String output = t.check(25);
    assertEquals("Divisible", output);
  }
}
```

🔥 27. CHECK PERFECT NUMBER

📄 perfecttest.java

```
package unittest;

public class perfecttest {
  public String check(int n){
    int sum = 0;

    for(int i = 1; i < n; i++){
      if(n % i == 0){
        sum += i;
      }
    }

    if(sum == n){
      return "Perfect";
    }
    else{
      return "Not Perfect";
    }
  }
}
```

📄 Perfecttestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Perfecttestcase extends TestCase {

  public void testPerfect() {
    perfecttest t = new perfecttest();
    String output = t.check(6);
    assertEquals("Perfect", output);
  }
}
```

🔥 28. CHECK CHARACTER TYPE

📄 chartypetest.java

```
package unittest;

public class chartypetest {
  public String check(char ch){
    if(Character.isDigit(ch)){
      return "Digit";
    }
    else if(Character.isLetter(ch)){
      return "Letter";
    }
    else{
      return "Special";
    }
  }
}
```

📄 Chartypetestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Chartypetestcase extends TestCase {

  public void testCharType() {
    chartypetest t = new chartypetest();
    String output = t.check('A');
    assertEquals("Letter", output);
```

```
  }
}
```

---

29. CHECK NUMBER IS MULTIPLE OF 3

📄 multiple3test.java

```
package unittest;

public class multiple3test {
  public String check(int n){
    if(n % 3 == 0){
      return "Multiple of 3";
    }
    else{
      return "Not Multiple of 3";
    }
  }
}
```

📄 Multiple3testcase.java

```
package unittest;

import junit.framework.TestCase;

public class Multiple3testcase extends TestCase {

  public void testMultiple3() {
    multiple3test t = new multiple3test();
    String output = t.check(9);
    assertEquals("Multiple of 3", output);
  }
}
```

🔥 30. FIND AVERAGE OF ARRAY

📄 averagetest.java

```
package unittest;

public class averagetest {
  public double avg(int arr[]){
    int sum = 0;

    for(int i = 0; i < arr.length; i++){
      sum += arr[i];
    }
```

```
    return (double) sum / arr.length;
  }
}
```

📄 Averagetestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Averagetestcase extends TestCase {

  public void testAverage() {
    averagetest t = new averagetest();
    int[] arr = {2,4,6,8};
    double output = t.avg(arr);
    assertEquals(5.0, output);
  }
}
```

🔥 31. CHECK STRING PALINDROME

📄 stringpaltest.java

```
package unittest;

public class stringpaltest {
  public String check(String str){
    String rev = "";

    for(int i = str.length()-1; i >= 0; i--){
      rev += str.charAt(i);
    }

    if(str.equals(rev)){
      return "Palindrome";
    }
    else{
      return "Not Palindrome";
    }
  }
}
```

📄 Stringpaltestcase.java

```
package unittest;

import junit.framework.TestCase;
```

```
public class Stringpaltestcase extends TestCase {

  public void testStringPalindrome() {
    stringpaltest t = new stringpaltest();
    String output = t.check("madam");
    assertEquals("Palindrome", output);
  }
}
```

🔥 32. CHECK GREATER THAN 100

📄 greater100test.java

```
package unittest;

public class greater100test {
  public String check(int n){
    if(n > 100){
      return "Greater";
    }
    else{
      return "Not Greater";
    }
  }
}
```

📄 Greater100testcase.java

```
package unittest;

import junit.framework.TestCase;

public class Greater100testcase extends TestCase {

  public void testGreater() {
    greater100test t = new greater100test();
    String output = t.check(150);
    assertEquals("Greater", output);
  }
}
```

🔥 33. FIND SECOND LARGEST (ARRAY)

📄 secondlargesttest.java

```
package unittest;
```

```
public class secondlargesttest {
  public int secondLargest(int arr[]){
    int first = arr[0];
    int second = arr[0];

    for(int i = 1; i < arr.length; i++){
      if(arr[i] > first){
        second = first;
        first = arr[i];
      }
      else if(arr[i] > second && arr[i] != first){
        second = arr[i];
      }
    }
    return second;
  }
}
```

📄 Secondlargesttestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Secondlargesttestcase extends TestCase {

  public void testSecondLargest() {
    secondlargesttest t = new secondlargesttest();
    int[] arr = {10, 20, 30, 40};
    int output = t.secondLargest(arr);
    assertEquals(30, output);
  }
}
```

🔥 34. CHECK NUMBER IS SQUARE NUMBER

📄 squarechecktest.java

```
package unittest;

public class squarechecktest {
  public String check(int n){
    int root = (int)Math.sqrt(n);

    if(root * root == n){
      return "Perfect Square";
    }
    else{
```

```
        return "Not Square";
      }
    }
}
```

📄 Squarechecktestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Squarechecktestcase extends TestCase {

  public void testSquare() {
    squarechecktest t = new squarechecktest();
    String output = t.check(16);
    assertEquals("Perfect Square", output);
  }
}
```

---

1. STUDENT RESULT SYSTEM

👉 Input: marks (0–100)
👉 Output: Grade + Result (Pass/Fail)

📄 studenttest.java

```
package unittest;

public class studenttest {
  public String result(int marks){

    if(marks < 0 || marks > 100){
      return "Invalid";
    }

    if(marks < 50){
      return "Fail";
    }
    else if(marks < 75){
      return "Pass - Grade C";
    }
    else if(marks < 90){
      return "Pass - Grade B";
    }
    else{
      return "Pass - Grade A";
    }
```

```
  }
}
```

📄 Studenttestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Studenttestcase extends TestCase {

  public void testStudent() {
    studenttest t = new studenttest();
    String output = t.result(85);
    assertEquals("Pass - Grade B", output);
  }
}
```

🔥 2. ATM WITHDRAWAL SYSTEM

👉 Conditions:

- Balance ≥ amount
- Amount multiple of 100

📄 atmtest.java

```
package unittest;

public class atmtest {
  public String withdraw(int balance, int amount){

    if(amount % 100 != 0){
      return "Invalid Amount";
    }

    if(amount > balance){
      return "Insufficient Balance";
    }

    return "Transaction Successful";
  }
}
```

📄 Atmtestcase.java

```
package unittest;

import junit.framework.TestCase;
```

```
public class Atmtestcase extends TestCase {

  public void testATM() {
    atmtest t = new atmtest();
    String output = t.withdraw(5000, 1000);
    assertEquals("Transaction Successful", output);
  }
}
```

🔥 3. LOGIN VALIDATION

👉 Conditions:

- Username = "admin"
- Password = "1234"

📄 logintest.java

```
package unittest;

public class logintest {
  public String login(String user, String pass){

    if(user.equals("admin") && pass.equals("1234")){
      return "Login Success";
    }
    else{
      return "Login Failed";
    }
  }
}
```

📄 Logintestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Logintestcase extends TestCase {

  public void testLogin() {
    logintest t = new logintest();
    String output = t.login("admin", "1234");
    assertEquals("Login Success", output);
  }
}
```

🔥 4. ELECTRICITY BILL

👉 Logic:

- First 100 units → ₹1/unit
- Next 100 → ₹2/unit
- Above 200 → ₹3/unit

📄 billtest.java

```
package unittest;

public class billtest {
  public int calculate(int units){

    if(units <= 100){
      return units * 1;
    }
    else if(units <= 200){
      return (100 * 1) + ((units - 100) * 2);
    }
    else{
      return (100 * 1) + (100 * 2) + ((units - 200) * 3);
    }
  }
}
```

📄 Billtestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Billtestcase extends TestCase {

  public void testBill() {
    billtest t = new billtest();
    int output = t.calculate(250);
    assertEquals(550, output);
  }
}
```

🔥 5. PASSWORD STRENGTH CHECK

👉 Conditions:

- Length ≥ 8
- Contains digit

📄 passwordtest.java

```
package unittest;

public class passwordtest {
  public String check(String pass){

    boolean hasDigit = false;

    for(int i = 0; i < pass.length(); i++){
      if(Character.isDigit(pass.charAt(i))){
        hasDigit = true;
        break;
      }
    }

    if(pass.length() >= 8 && hasDigit){
      return "Strong";
    }
    else{
      return "Weak";
    }
  }
}
```

📄 Passwordtestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Passwordtestcase extends TestCase {

  public void testPassword() {
    passwordtest t = new passwordtest();
    String output = t.check("abc12345");
    assertEquals("Strong", output);
  }
}
```

---

6. BANK LOAN ELIGIBILITY

👉 Conditions:

- Age ≥ 21
- Salary ≥ 20000

📄 loantest.java

```
package unittest;

public class loantest {
  public String check(int age, int salary){

    if(age < 21){
      return "Not Eligible";
    }

    if(salary < 20000){
      return "Not Eligible";
    }

    return "Eligible";
  }
}
```

📄 Loantestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Loantestcase extends TestCase {

  public void testLoan() {
    loantest t = new loantest();
    String output = t.check(25, 30000);
    assertEquals("Eligible", output);
  }
}
```

🔥 7. DISCOUNT CALCULATION

👉 Logic:

- ≥5000 → 20%
- ≥2000 → 10%
- else → 5%

📄 discounttest.java

```
package unittest;

public class discounttest {
  public double calculate(double amount){

    if(amount >= 5000){
```

```
        return amount * 0.8;
      }
      else if(amount >= 2000){
        return amount * 0.9;
      }
      else{
        return amount * 0.95;
      }
    }
}
```

📄 Discounttestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Discounttestcase extends TestCase {

    public void testDiscount() {
      discounttest t = new discounttest();
      double output = t.calculate(5000);
      assertEquals(4000.0, output);
    }
}
```

🔥 8. TRAFFIC SIGNAL SYSTEM

👉 Input: color
👉 Output: action

📄 traffictest.java

```
package unittest;

public class traffictest {
    public String action(String color){

      if(color.equals("red")){
        return "Stop";
      }
      else if(color.equals("yellow")){
        return "Wait";
      }
      else if(color.equals("green")){
        return "Go";
      }
      else{
```

```
        return "Invalid";
      }
    }
}
```

📄 Traffictestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Traffictestcase extends TestCase {

    public void testTraffic() {
      traffictest t = new traffictest();
      String output = t.action("green");
      assertEquals("Go", output);
    }
}
```

🔥 9. EMPLOYEE BONUS SYSTEM

👉 Logic:

- ≥5 years → 20% bonus
- ≥2 years → 10%
- else → no bonus

📄 bonustest.java

```
package unittest;

public class bonustest {
    public double bonus(double salary, int years){

      if(years >= 5){
        return salary * 0.2;
      }
      else if(years >= 2){
        return salary * 0.1;
      }
      else{
        return 0;
      }
    }
}
```

📄 Bonustestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Bonustestcase extends TestCase {

  public void testBonus() {
    bonustest t = new bonustest();
    double output = t.bonus(10000, 5);
    assertEquals(2000.0, output);
  }
}
```

🔥 10. MOBILE NUMBER VALIDATION

👉 Conditions:

- Exactly 10 digits
- Starts with 6–9

📄 mobiletest.java

```
package unittest;

public class mobiletest {
  public String check(String num){

    if(num.length() != 10){
      return "Invalid";
    }

    char first = num.charAt(0);

    if(first >= '6' && first <= '9'){
      return "Valid";
    }
    else{
      return "Invalid";
    }
  }
}
```

📄 Mobiletestcase.java

```
package unittest;

import junit.framework.TestCase;
```

```
public class Mobiletestcase extends TestCase {

  public void testMobile() {
    mobiletest t = new mobiletest();
    String output = t.check("9876543210");
    assertEquals("Valid", output);
  }
}
```

🔥 11. VOTING ELIGIBILITY

👉 Conditions:

- Age ≥ 18

📄 votetest.java

```
package unittest;

public class votetest {
  public String check(int age){

    if(age >= 18){
      return "Eligible";
    }
    else{
      return "Not Eligible";
    }
  }
}
```

📄 Votetestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Votetestcase extends TestCase {

  public void testVote() {
    votetest t = new votetest();
    String output = t.check(20);
    assertEquals("Eligible", output);
  }
}
```

🔥 12. SHOPPING CART TOTAL WITH TAX

👉 Logic:

- Add 18% GST

📄 carttest.java

```
package unittest;

public class carttest {
  public double total(double amount){
    return amount + (amount * 0.18);
  }
}
```

📄 Carttestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Carttestcase extends TestCase {

  public void testCart() {
    carttest t = new carttest();
    double output = t.total(1000);
    assertEquals(1180.0, output);
  }
}
```

---

1. REVERSE STRING

📄 reversestringtest.java

```
package unittest;

public class reversestringtest {
  public String reverse(String str){
    String rev = "";

    for(int i = str.length() - 1; i >= 0; i--){
      rev = rev + str.charAt(i);
    }
    return rev;
  }
}
```

📄 Reversestringtestcase.java

```
package unittest;
```

```
import junit.framework.TestCase;

public class Reversestringtestcase extends TestCase {

  public void testReverseString() {
    reversestringtest t = new reversestringtest();
    String output = t.reverse("hello");
    assertEquals("olleh", output);
  }
}
```

🔥 2. STRING PALINDROME

📄 stringpalindrometest.java

```
package unittest;

public class stringpalindrometest {
  public String check(String str){
    String rev = "";

    for(int i = str.length() - 1; i >= 0; i--){
      rev = rev + str.charAt(i);
    }

    if(str.equals(rev)){
      return "Palindrome";
    }
    else{
      return "Not Palindrome";
    }
  }
}
```

📄 Stringpalindrometestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Stringpalindrometestcase extends TestCase {

  public void testStringPalindrome() {
    stringpalindrometest t = new stringpalindrometest();
    String output = t.check("madam");
    assertEquals("Palindrome", output);
  }
}
```

🔥 3. NUMBER PALINDROME

📄 numberpalindrometest.java

```
package unittest;

public class numberpalindrometest {
  public String check(int n){
    int temp = n, rev = 0;

    while(n > 0){
      int r = n % 10;
      rev = rev * 10 + r;
      n = n / 10;
    }

    if(temp == rev){
      return "Palindrome";
    }
    else{
      return "Not Palindrome";
    }
  }
}
```

📄 Numberpalindrometestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Numberpalindrometestcase extends TestCase {

  public void testNumberPalindrome() {
    numberpalindrometest t = new numberpalindrometest();
    String output = t.check(121);
    assertEquals("Palindrome", output);
  }
}
```

🔥 4. REVERSE NUMBER

📄 reversenumbertest.java

```
package unittest;

public class reversenumbertest {
```

```
  public int reverse(int n){
    int rev = 0;

    while(n > 0){
      rev = rev * 10 + (n % 10);
      n = n / 10;
    }
    return rev;
  }
}
```

📄 Reversenumbertestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Reversenumbertestcase extends TestCase {

  public void testReverseNumber() {
    reversenumbertest t = new reversenumbertest();
    int output = t.reverse(123);
    assertEquals(321, output);
  }
}
```

🔥 5. COUNT WORDS IN STRING

📄 wordcounttest.java

```
package unittest;

public class wordcounttest {
  public int count(String str){
    String[] words = str.split(" ");
    return words.length;
  }
}
```

📄 Wordcounttestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Wordcounttestcase extends TestCase {

  public void testWordCount() {
    wordcounttest t = new wordcounttest();
```

```
    int output = t.count("I love coding");
    assertEquals(3, output);
  }
}
```

🔥 6. REMOVE VOWELS FROM STRING

📄 removevowelstest.java

```
package unittest;

public class removevowelstest {
  public String remove(String str){
    return str.replaceAll("[aeiouAEIOU]", "");
  }
}
```

📄 Removevowelstestcase.java

```
package unittest;

import junit.framework.TestCase;

public class Removevowelstestcase extends TestCase {

  public void testRemoveVowels() {
    removevowelstest t = new removevowelstest();
    String output = t.remove("hello");
    assertEquals("hll", output);
  }
}
```

🔥 7. COUNT CHARACTERS IN STRING

📄 charcounttest.java

```
package unittest;

public class charcounttest {
  public int count(String str){
    return str.length();
  }
}
```

📄 Charcounttestcase.java

```
package unittest;
```

```
import junit.framework.TestCase;

public class Charcounttestcase extends TestCase {

   public void testCharCount() {
      charcounttest t = new charcounttest();
      int output = t.count("hello");
      assertEquals(5, output);
   }
}
```